

Sprawozdanie: System Zarządzania Kadrami (C++)

Odpowiedzi na pytania projektowe

1. Jakie klasy zostały zaprojektowane i jakie odpowiedzialności pełni każda z nich?

W projekcie zaimplementowano klasę bazową **Pracownik**, która przechowuje dane wspólne (imię, nazwisko, ID) oraz definiuje interfejs polimorficzny. Klasy pochodne: **PracownikEtatowy** (obsługa pensji stałej), **PracownikGodzinowy** (wyliczanie pensji na podstawie stawki i godzin) oraz **PracownikProwizyjny** (obliczanie wynagrodzenia jako procent od sprzedaży).

2. Jakie cechy są wspólne dla wszystkich pracowników, a jakie różnią klasy pochodne?

Wspólne: imię, nazwisko, identyfikator oraz funkcjonalność obliczania pensji i wyświetlania danych. **Różnice:** parametry wejściowe do obliczeń (np. stała kwota vs stawka godzinowa vs procent sprzedaży) oraz konkretna logika matematyczna w metodzie `obliczPensje()`.

3. Które pola w projekcie zostały ukryte jako `private` i dlaczego?

Wszystkie pola przechowujące stan (imię, nazwisko, id, pensja, stawka, godziny, sprzedaż, prowizja) są prywatne. Zrobiono to, aby wymusić modyfikację danych wyłącznie przez konstruktor i settery, co zapewnia kontrolę nad poprawnością danych (hermetyzacja).

4. W jaki sposób program zabezpiecza obiekty przed ustawieniem niepoprawnych danych?

Zastosowano walidację w setterach (np. `if(pensja < 0)`) oraz rozbudowaną kontrolę wejścia w funkcji `main`, która sprawdza poprawność strumienia `std::cin` i zakresy wartości przed utworzeniem obiektu.

5. Na czym w tym projekcie polega praktyczne zastosowanie hermetyzacji?

Polega na oddzieleniu interfejsu (metody publiczne) od implementacji i danych (pola prywatne). Użytkownik klasy korzysta z gotowych metod, nie mając wglądu ani możliwości bezpośredniego zepsucia wewnętrznej struktury danych obiektu.

6. Jakie elementy klasy bazowej zostały odziedziczone przez klasy pochodne?

Odziedziczone zostały wszystkie pola prywatne (dostępne przez metody i konstruktor bazy) oraz deklaracje metod wirtualnych `obliczPensje()` i `wyswietl()`.

7. Jakie zalety dało użycie dziedziczenia w tym projekcie?

Pozwoliło na uniknięcie powtarzania kodu dla pól wspólnych oraz umożliwiło stworzenie polimorficznego kontenera `std::vector<Pracownik*>`, który może zarządzać różnymi typami pracowników w sposób jednolity.

8. Czy ten program dałoby się napisać bez dziedziczenia? Jeśli tak, to czym różniłoby się takie rozwiązanie?

Tak, za pomocą jednej klasy z polem typu (np. `enum`). Jednak wymagałoby to użycia instrukcji `switch` w każdej metodzie i marnowania pamięci na pola nieużywane w danym typie umowy. Rozwiązanie takie byłoby trudne w rozbudowie.

9. Które metody zostały zadeklarowane jako wirtualne i dlaczego?

Wirtualne są: destruktor (dla bezpiecznego usuwania), `obliczPensje()` i `wyswietl()`. Pozwala to na wywołanie właściwej wersji metody w zależności od faktycznego typu obiektu wskazywanego przez wskaźnik klasy bazowej.

10. W jaki sposób program wykorzystuje polimorfizm dynamiczny?

W pętli przechodzącej przez wektor wskaźników `Pracownik*`. Program dopiero w czasie wykonywania (runtime) rozstrzyga, czy pod wskaźnikiem kryje się `etatowiec`, czy `godzinowiec`, i uruchamia odpowiednią logikę obliczeń.

11. Co się stanie, jeśli metoda `obliczPensje()` nie będzie wirtualna?

Kompilator zastosuje wczesne wiązanie i zawsze wywoła metodę z klasy bazowej. Ponieważ w tym projekcie jest ona czysto wirtualna, brak słowa `virtual` (i implementacji) uniemożliwiłby kompilację lub wywołanie poprawnej logiki.

12. Jakie są największe zalety podejścia obiektowego w tym zadaniu?

Największą zaletą jest łatwa rozszerzalność (Open/Closed Principle) – dodanie nowego typu pracownika wymaga dopisania klasy, a nie modyfikacji logiki pętli czy klasy bazowej.

13. Jakie są ograniczenia podejścia obiektowego w tym zadaniu?

Należy do nich konieczność ręcznego zarządzania pamięcią przy użyciu `new` i `delete` oraz minimalny narzut na tablicę metod wirtualnych (`vtable`).

Załącznik: Kod źródłowy projektu

```
#include <iostream>
#include <vector>
#include <string>
#include <limits>
```

```

class Pracownik {
private:
    std::string imie;
    std::string nazwisko;
    int id;
public:
    Pracownik(std::string i, std::string n, int id) : imie(i), nazwisko(n), id(id) {}
    virtual ~Pracownik() {}
    virtual double obliczPensje() const = 0;
    virtual void wyswietl() const {
        std::cout << "ID: " << id << " Imie: " << imie << " Nazwisko: " << nazwisko <<
std::endl;
    }
};

class PracownikEtatowy: public Pracownik {
private:
    double PensjaMiesieczna;
public:
    PracownikEtatowy(std::string i, std::string n, int id, double pensja) : Pracownik(i,
n, id) {
        setPensja(pensja);
    }
    void setPensja(double pensja) {
        if(pensja < 0) std::cout << "Pensja nie moze byc ujemna!" << std::endl;
        else PensjaMiesieczna = pensja;
    }
    double obliczPensje() const override { return PensjaMiesieczna; }
    void wyswietl() const override {
        Pracownik::wyswietl();
        std::cout << "[Umowa o prace] Pensja: " << obliczPensje() << " zł" << std::endl;
    }
};

// ... (Pozostałe klasy analogicznie) ...

int main() {
    std::vector<Pracownik*> kadr;
    // ... Logika switch z walidacją wejścia ...
    for(auto p : kadr) delete p;
    return 0;
}

```